Models of Computation: Languages, Machines & Logic

01 / Finite automata → 02 / Grammars and parsing → 03 / Computability → 04 / Logic and inference

01 / Three empty objects

ε is the empty string; {ε} is the language containing it; ∅ is the language with no strings. Every Kleene star includes ε.

02 / DFA design

Name each state by what it says about the processed prefix. Prove initialization and preservation, then connect the final state to acceptance after all input is consumed.

03 / NFA acceptance

One successful complete run suffices. Epsilon moves consume nothing. Swapping NFA final states does not complement its language; first determinize and complete the machine.

04 / Subset construction

Start with epsilon closure. For each symbol, union all possible moves and close again. Accept subsets intersecting F. At most 2^m subsets arise from m NFA states.

05 / Regular equivalence

Build a product DFA accepting disagreement, then test accepting-state reachability. Empty symmetric difference means equal languages; a reachable disagreement path supplies a counterexample.

06 / CNF and CYK

Grammar CNF uses A→BC or A→a, with a protected start epsilon exception. CYK processes increasing spans; T[i,j] means w[i:j], and splits satisfy i<k<j.

07 / Recognize versus decide

A recognizer accepts every member but may loop outside its language. A decider must halt on every input. A timeout alone establishes neither rejection nor undecidability.

08 / Complement theorem

L is decidable exactly when L and its complement are recognizable. Alternate the two recognizers fairly; whichever accepts determines the correct terminating answer.

09 / Three logical classifications

Satisfiable means true somewhere; valid means true everywhere; unsatisfiable means true nowhere. With k atoms, exhaustive truth-table testing uses 2^k assignments.

10 / Entailment test

Γ entails F exactly when the premises together with ¬F are unsatisfiable. A countermodel makes every premise true and the conclusion false.

11 / Quantifier discipline

All A are B: ∀x(A(x)→B(x)). Some A are B: ∃x(A(x)∧B(x)). ¬∀xP≡∃x¬P and ¬∃xP≡∀x¬P, so "not all A are B" is ∃x(A(x)∧¬B(x)); mixed quantifier order controls witness dependence.

12 / Impossibility proofs

Assume a machine exists, then force a contradiction using repeated states or self-reference. One nonterminating implementation does not prove undecidability; the claim must exclude every possible decider.